

CASO DE ESTUDIO: GUARDIAN OF MISSING

DOCUMENTO DE INGENIERÍA DE SOFTWARE - UNIDAD I: DEFINICIÓN DEL PROCESO DE DESARROLLO WEB

Carrera: Ingeniería en Desarrollo y Gestión de Software

Asignatura: Desarrollo Web Integral (Noveno Cuatrimestre)

Proyecto: Ecosistema de Seguridad Personal "Guardian of Missing"

Equipo de Desarrollo: Equipo MR (Matías, Mauricio, Erick, José Arturo)

CAPÍTULO I: PLAN DEL PROCESO DE DESARROLLO (METODOLOGÍA ÁGIL)

1.1 Selección de la Metodología Ágil: Scrum

Para el desarrollo del ecosistema integral "Guardian of Missing", se ha seleccionado la metodología ágil **Scrum**. Esta elección se justifica debido a la naturaleza crítica, adaptativa y multiplataforma del proyecto, el cual involucra una aplicación móvil, una aplicación para smartwatch y una API web centralizadora. Dado que existen puntos pendientes de definición técnica complejos —como la frecuencia del rastreo GPS (polling) y la viabilidad de alarmas en altavoces de los wearables—, Scrum permite realizar entregas parciales y funcionales (Incrementos) al finalizar cada ciclo de trabajo (Sprint), facilitando la reevaluación constante de requerimientos y la mitigación de riesgos técnicos tempranos sin detener el avance general del proyecto.

1.2 Planificación de Sprints y Cronograma Inicial

El desarrollo se estructurará en 4 Sprints de dos semanas cada uno, acumulando el total de horas prácticas y teóricas estipuladas para el análisis y cimentación del sistema.

Sprint	Objetivo Principal	Entregables y Componentes Incluidos
Sprint 1	Configuración de Base y Arquitectura Central	Modelado de datos en MySQL y MongoDB,

Sprint	Objetivo Principal	Entregables y Componentes Incluidos
		configuración del entorno base con FastAPI, definición del esquema de autenticación e inicialización del repositorio Git (Git Flow).
Sprint 2	Módulo 1: Personas Vulnerables y Ubicación	Implementación de geocercas, lógica de rastreo permanente (24/7), procesamiento de mapas de calor de zonas de riesgo y lógica de notificaciones de riesgo enviadas a smartwatches.
Sprint 3	Módulo 2: Botón de Pánico y Evidencia	Lógica del Botón de Pánico en Backend, envío de alertas a contactos de confianza, activación silenciosa remota de grabación de audio/video y control de cámara.
Sprint 4	Pruebas, Contenedores y Liberación	Automatización de pruebas unitarias y de integración, empaquetado en contenedores Docker, despliegue en infraestructura de cómputo en la nube e implementación de certificados TLS/SSL.

1.3 Roles y Responsabilidades (Equipo MR)

- **Matías (Scrum Master / Backend Engineer):** Responsable de garantizar el cumplimiento del marco Scrum, mitigar bloqueos del equipo y liderar el diseño del backend asíncrono e integraciones de bases de datos.
- **Mauricio (Product Owner / QA Specialist):** Encargado de la gestión del Product Backlog, validación de reglas de negocio de los reportes y el diseño del esquema automatizado de pruebas.
- **Erick (Frontend & Mobile Developer):** Responsable de la maquetación e interfaces adaptativas en dispositivos móviles, lógica de consumo de API en la aplicación móvil y sincronización con el mapa.

- **José Arturo (Wearable Integration Engineer):** Encargado exclusivo de la lógica de la aplicación para Smartwatch, optimización del consumo de batería y la comunicación bidireccional mediante sockets/Bluetooth hacia el Core móvil.

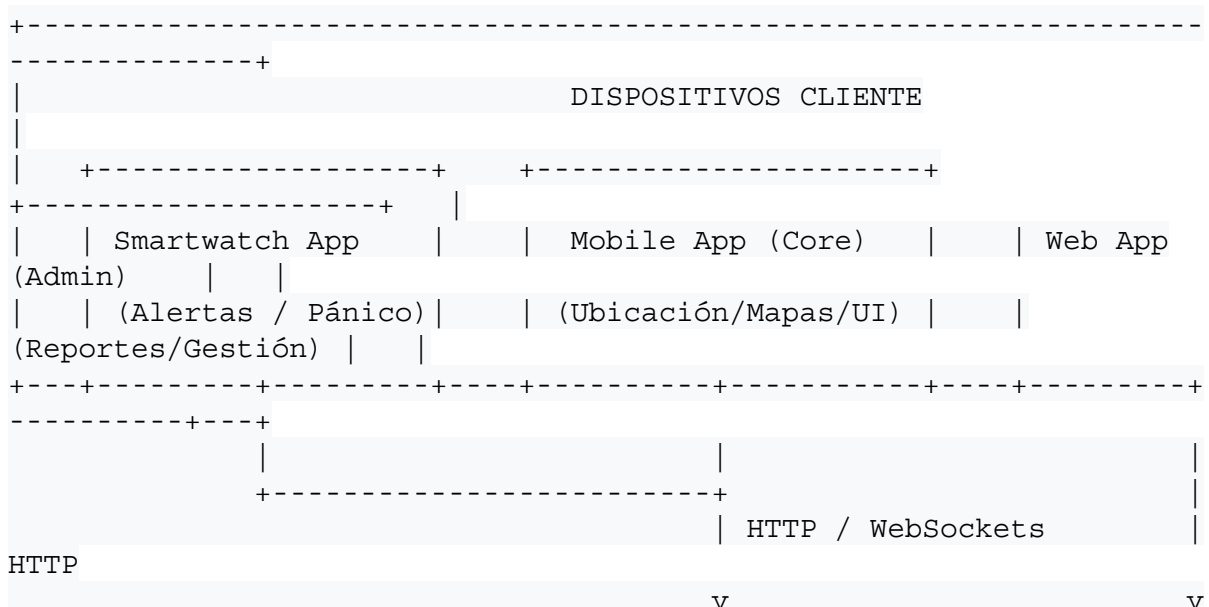
CAPÍTULO II: ESPECIFICACIÓN Y ARQUITECTURA DEL SOFTWARE

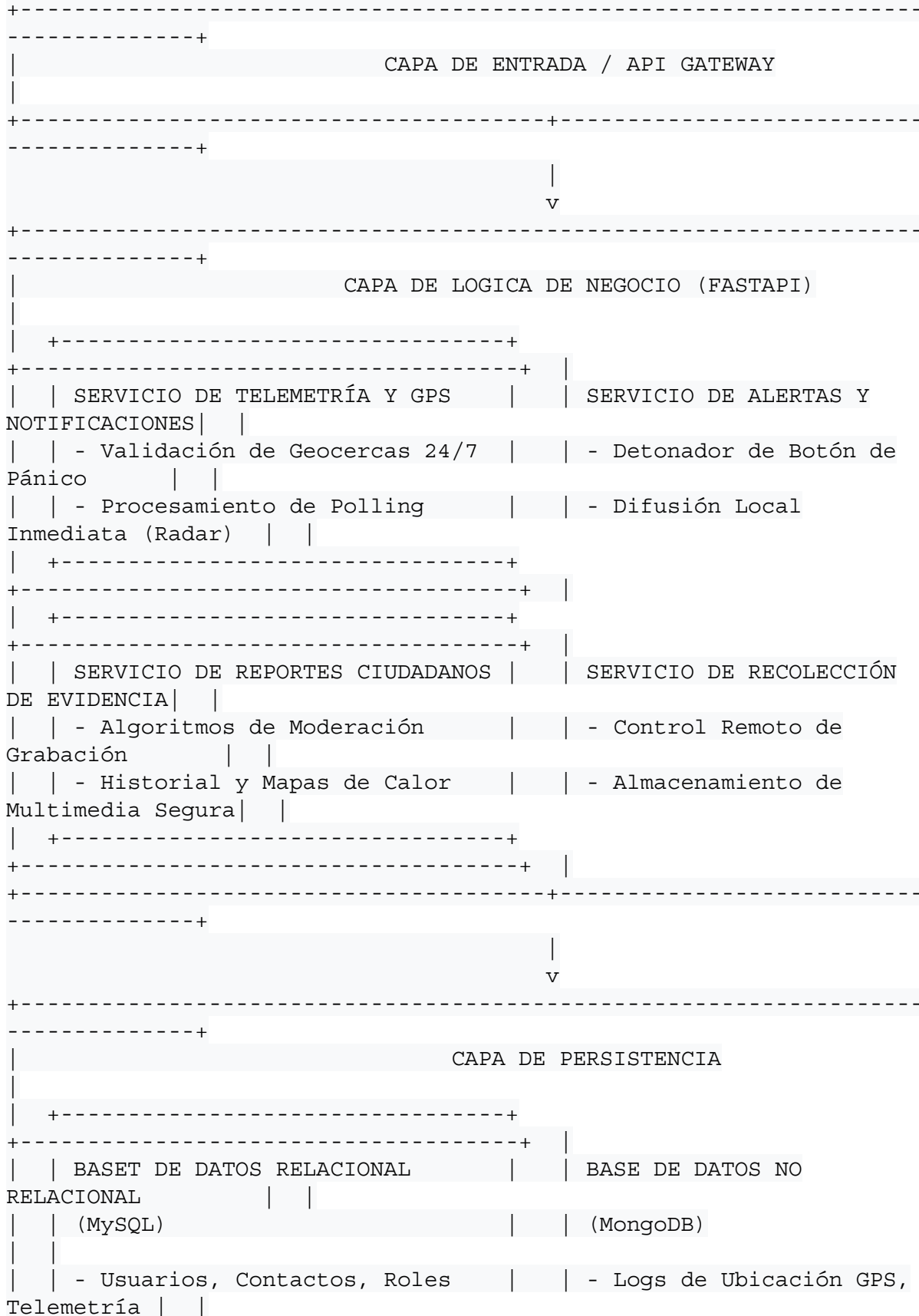
2.1 Justificación de la Arquitectura Seleccionada

Para "Guardian of Missing" se ha adoptado un modelo arquitectónico de **Microservicios e interfaces basadas en API RESTful asíncronas**, complementado internamente con una **Arquitectura Limpia (Clean Architecture)** en el núcleo del backend web. Esta decisión responde directamente a los atributos de calidad requeridos:

- **Disponibilidad y Concurrencia de Alta Densidad:** El sistema recibirá constantes ráfagas de datos de ubicación GPS procedentes de los dispositivos del Módulo 1. Separar la ingesta de telemetría de ubicación de la capa de visualización web de reportes garantiza que un colapso en las consultas web no impida la recepción de un Botón de Pánico crítico.
- **Escalabilidad Multiplataforma:** Al centralizar las reglas de negocio en servicios desacoplados, la app móvil, el reloj y la plataforma web consumen los mismos endpoints, agilizando la integración y el mantenimiento.

2.2 Esquema Conceptual de la Arquitectura del Sistema





```

| | - Credenciales e Historiales | | - Reportes Anónimos y
Mapas de Calor| |
| +-----+
+-----+ |
+-----+
-----+

```

CAPÍTULO III: PROPUESTA DE PATRONES DE DISEÑO

Para resolver los problemas específicos planteados en las reglas de negocio de la aplicación, se incorporan tres patrones de diseño de software fundamentales, cumpliendo rigurosamente la directiva de emplear un patrón creacional, uno estructural y uno de comportamiento.

3.1 Patrón Creacional: Factory Method

- **Problema a resolver:** El sistema maneja diferentes variantes de eventos de seguridad y reportes que poseen comportamientos, flujos de validación y ciclos de vida divergentes (Reporte de Persona Desaparecida del Módulo 1 frente a Reporte Ciudadano Anónimo del Módulo 2).
- **Solución en el proyecto:** Se implementará una clase abstracta CreadorDeReportes con un método de fabricación. Dependiendo del contexto o la carga útil recibida en el endpoint de la API, las subclases concretas instanciarán un ReporteModulo1 (que requiere vincular metadatos de permisos fijos de GPS y validación de identidad) o un ReporteModulo2 (que activa automáticamente los Algoritmos de Moderación y filtros contra spam/difamación antes de guardarse en MongoDB).

3.2 Patrón Estructural: Facade (Fachada)

- **Problema a resolver:** La activación del Botón de Pánico desde un Smartwatch o teléfono celular requiere la orquestación coordinada de múltiples subsistemas complejos del Backend de manera atómica: persistir la alerta en MySQL, recuperar los tokens de notificaciones push de los contactos de confianza, consultar las coordenadas de los usuarios locales en un rango cercano e inicializar la transmisión del buffer para la Grabación Silenciosa.
- **Solución en el proyecto:** Se diseña la clase fachada FachadaEmergenciaWeb. El controlador de la API solo realiza una llamada simple a FachadaEmergenciaWeb.dispararAlertaCompleta(userId, coordenada). Esta fachada se

encarga internamente de comunicarse de manera transparente con el Servicio de Notificaciones, el Servicio de Grabación y el Buscador Espacial de Coordenadas, aislando la complejidad técnica de los clientes.

3.3 Patrón de Comportamiento: Observer (Observador)

- **Problema a resolver:** Cuando se vulnera una Geocerca del Módulo 1 (un adulto mayor o paciente con Alzheimer abandona el rango seguro), múltiples entidades interesadas (la aplicación del cuidador principal, el centro de monitoreo web y la base de datos distribuida) deben reaccionar inmediatamente en tiempo real sin que el módulo de rastreo GPS esté acoplado directamente a ellas.
- **Solución en el proyecto:** El motor de Geocercas actuará como el **Sujeto (Subject)**. Diversos componentes se registrarán como **Observadores (Observers)**, tales como el ServicioNotificacionesPush y el ServicioRegistroAuditoria. En el momento exacto en que la telemetría rompe el polígono configurado, el Sujeto ejecuta un bucle de notificación automática, actualizando en milisegundos las interfaces de los cuidadores y activando los protocolos correspondientes.

CAPÍTULO IV: SELECCIÓN Y JUSTIFICACIÓN DE FRAMEWORKS

4.1 Framework de Backend: FastAPI (Python)

Para el núcleo de servicios web integrales se ha seleccionado **FastAPI** sobre otras alternativas basadas en lenguajes tradicionales. Los fundamentos técnicos de esta selección son:

- **Soporte Asíncrono Nativo (async/await):** Fundamental para gestionar eficientemente las peticiones concurrentes de ubicación en tiempo real (polling continuo) sin bloquear los hilos del servidor de aplicaciones.
- **Validación de Datos con Pydantic:** Permite estructurar de forma estricta los esquemas JSON de intercambio de datos (como las reglas de la última ubicación conocida), rechazando peticiones malformadas de inmediato y garantizando seguridad en la capa de transporte.
- **Rendimiento Elevado:** Al estar construido sobre Starlette y Uvicorn, ofrece velocidades equiparables a NodeJS y Go, ideales para un ecosistema de seguridad de respuesta inmediata.

4.2 Enfoque Híbrido de Almacenamiento: MySQL y MongoDB

- **MySQL:** Se utilizará para gestionar los datos de alta integridad referencial del ecosistema, como tablas de usuarios, roles de seguridad, credenciales hash, emparejamiento de dispositivos y relaciones estrictas de contactos de confianza y asignación de cuidadores.
- **MongoDB:** Su naturaleza no relacional orientada a documentos es perfecta para almacenar el volumen masivo de logs e históricos de ubicación GPS generados 24/7 por el Módulo 1. Asimismo, permite estructurar de forma flexible los reportes ciudadanos anónimos del Módulo 2 y procesar de manera optimizada los índices geoespaciales para renderizar los mapas de calor de zonas de riesgo.

CAPÍTULO V: ESQUEMA Y ESTRATEGIA DE PRUEBAS

El esquema de pruebas para la validación del proceso web asegura la estabilidad antes de la liberación en producción, dividiéndose en cuatro niveles tácticos operados mediante herramientas de automatización.

Tipo de Prueba	Objetivo Tecnológico	Estrategia / Caso Práctico en Guardian of Missing
Pruebas Unitarias	Validar el comportamiento aislado de funciones de código.	Verificar de forma matemática que el algoritmo de Geocercas dispare correctamente un booleano positivo cuando una coordenada GPS simulada se encuentre fuera del radio límite establecido.
Pruebas de Integración	Comprobar la intercomunicación entre módulos y APIs.	
Pruebas de Caja Negra	Validar flujos funcionales completos desde la perspectiva del usuario final sin inspeccionar código.	Verificar que al enviar una petición POST al endpoint del Botón de Pánico, el sistema guarde el registro en MySQL y dispare de forma efectiva los tokens hacia Firebase Cloud Messaging (FCM).
Pruebas de Seguridad	Garantizar la protección de datos personales e integridad	Simular peticiones anónimas sin cabecera Bearer Token

Tipo de Prueba	Objetivo Tecnológico	Estrategia / Caso Práctico en Guardian of Missing
	de endpoints.	(JWT) hacia los endpoints privados de rastreo para certificar el bloqueo estricto del middleware de autenticación de FastAPI.

CAPÍTULO VI: REPORTE DE CONFIGURACIÓN DEL ENTORNO DE DESARROLLO

6.1 Lista de Herramientas del Entorno

- **Lenguaje de Programación:** Python (Versión 3.10 o superior)
- **Entorno Virtual de Python:** venv o Poetry para el aislamiento riguroso de dependencias.
- **Sistemas Gestores de Bases de Datos:** MySQL Server 8.0 y MongoDB Community Server 6.0 localmente para desarrollo.
- **Herramienta de Control de Versiones:** Git, configurado globalmente en las estaciones de trabajo del Equipo MR.
- **Cliente de Pruebas de API:** Postman o Insomnia para la simulación de payloads JSON de emergencia.

6.2 Parámetros de Configuración del Entorno (Archivo de Variables de Entorno .env)

Para garantizar portabilidad y seguridad, todas las credenciales se inyectan en el runtime de FastAPI mediante variables de entorno del sistema operativo. A continuación se define la estructura paramétrica del archivo de configuración requerido para el despliegue del entorno local de desarrollo:

```
# --- CONFIGURACIÓN DEL SERVIDOR WEB ---
APP_ENV=development
SECRET_KEY=9a82f3b4c5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2
ACCESS_TOKEN_EXPIRE_MINUTES=60

# --- PERSISTENCIA RELACIONAL (MYSQL - EQUIPO MR) ---
MYSQL_HOST=127.0.0.1
```



```
MYSQL_PORT=3306
MYSQL_USER=mr_dev_user
MYSQL_PASSWORD=DB_Secure_Pass_2026
MYSQL_DATABASE=guardian_missing_relational

# --- PERSISTENCIA NO RELACIONAL / TELEMETRÍA (MONGODB) ---
MONGO_URI=mongodb://mr_dev_user:Mongo_Secure_Pass_2026@127.0.0.1:270
17/
MONGO_DATABASE=guardian_missing_telemetry

# --- PARAMETRIZACIÓN DE REGLAS DE NEGOCIO ---
DEFAULT_POLLING_INTERVAL_SECONDS=60
MAX_GEOCENCE_RADIUS_METERS=5000
MODERATION_ALGORITHM_THRESHOLD=0.75

# --- SERVICIOS DE INFRAESTRUCTURA Y COMPONENTES EXTERNOS ---
FIREBASE_API_KEY=AiZaSyA1B2C3D4E5F6G7H8I9J0K1L2M3N4O5P6
SSL_CERT_PATH=/etc/ssl/certs/guardian_missing_dev.crt
```